

An analytic study of architecture, security, privacy, query processing, and performance evaluation of database-as-a-service

Fakhri Alam Khan¹ | Mona Jamjoom² | Awais Ahmad³ | Muhammad Asif³ 

¹Institute of Management Sciences,
Peshawar Pakistan

²Department of Computer Sciences,
College of Computer and Information
Sciences, Princess Nourah Bint
Abdulrahman University Riyadh, Saudi
Arabia

³Department of Computer Science,
National Textile University, Faisalabad,
Pakistan

Correspondence

Muhammad Asif, Department of
Computer Science, National Textile
University, Faisalabad 37610, Pakistan.
Email: asif@ntu.edu.pk

Abstract

In the present era of technology, the pace of data generation is very high, and the biggest challenge is the manageability of data with efficient and secure storage options as well as at lower cost. Therefore, in this article, we have elaborated the database-as-a-service (DaaS) concept having its introduction, architecture, security and privacy, query processing, and an analytic performance evaluation relating to query performance on encrypted data of DaaS. We have conducted an analytic study of the state-of-the-art by analyzing CPU and query execution time. According to this evaluation, it is concluded that data encryption and decryption need heavy processing power and are time-consuming. Moreover, there is several query execution problems that exist on encrypted data. A major improvement in query performance may be achieved through parallel query execution after the deployment of multithreading having maximum possible threads.

1 | INTRODUCTION

Data is an essential asset for any organization that is growing day by day. From tiny devices to megastructures, every entity in this world is generating data, whether it is living or nonliving. Data may be comprised of two types. It may be structured or unstructured. Structured data may be fingered out easily since it has prearranged data types, structure, and correct format, although unstructured data has no clear structure because unstructured data may have text documents, images, audio, videos, and images.¹⁻³

Some decades ago, data was stored in file systems, which were not an efficient way and arose multiple problems like redundancy, atomicity, inconsistency, data access, and isolation problems. Structured data may be processed easily by DBMS. An advanced form of DBMS is RDBMS, through which we can manage and create relationships of multiple entities that deliver robust control over the aforementioned issues, especially for redundancy problems of data. Structured Query Language (SQL) is not a programming language but works as a nonprocedural language. It is used in database management systems. It is utilized to store and extract data. SQL entails three types of categories. DML is used to extract or store data from the database schema, whereas DDL is usually used to produce database schemas as tables, although DCL controls the ownerships of DBMS.^{4,5}

Database-as-a-service (DaaS)^{6,7} actually saves the main cost in the shape of software and hardware as well as skilled professionals. DaaS creates a good attraction for mega organizations because major tasks like backup and recovery of the database also skipped and added to the duty of DaaS service providers (SPs). Furthermore, DaaS also adds further benefits like database deployment as well as 24/7 connectivity.

DaaS helps to get rid of these above-discussed tasks, which ultimately saves high cost. In DaaS, we actually rent out services of hardware and software as well as technical staff. Data owners (DOs) are the users who buy the services from

SPs. They store their data to remote servers of DaaS. SPs typically use RDBMS to store the data of DOs. Oracle, SQL Server, and IBM DB2 are prime examples of RDBMS.

DOs get major benefits like manageability of backup, database reorganization, and database upgrade. SPs also provide application interfaces for DOs where they can store and process their data by using queries after ensuring Internet connectivity.

Although DaaS provides many benefits, there are also some challenges that need to be addressed to gain prime benefits from DaaS architecture. These challenges are data security and query performance. In this review, we are going to conduct an analytical study on the state-of-the-art proposals in which our major concern is query performance, which will be critically analyzed upon CPU and query execution time.

This paper continues from the second section in which different architectures of DaaS are explained. Then, in the third section, different proposals for security and privacy in DaaS are elaborated. Afterward, we have discussed the query processing mechanism of DaaS in the fourth section. Afterward, an analytical evaluation of state-of-the-art related to query performance in DaaS is analyzed in the fifth section. Finally, we conclude this paper, and some limitations and future works are also discussed.

2 | DaaS ARCHITECTURE

The DaaS architecture is very simple enough because DOs simply have to hire services of SPs because SPs provide different types of services according to the specification of their own custom-built DaaS-based platform architecture. DOs typically use custom-built application that developed from freelancers for the security of data, or they may also purchase services from SPs. Figure 1 is showing the simplest architecture of a DaaS.⁸

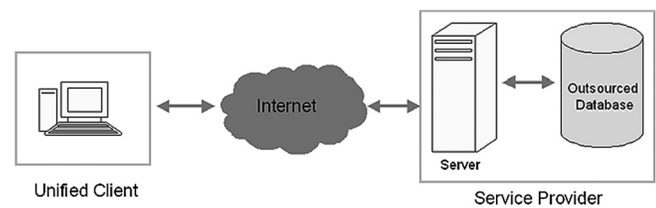


FIGURE 1 Simple DaaS architecture⁸

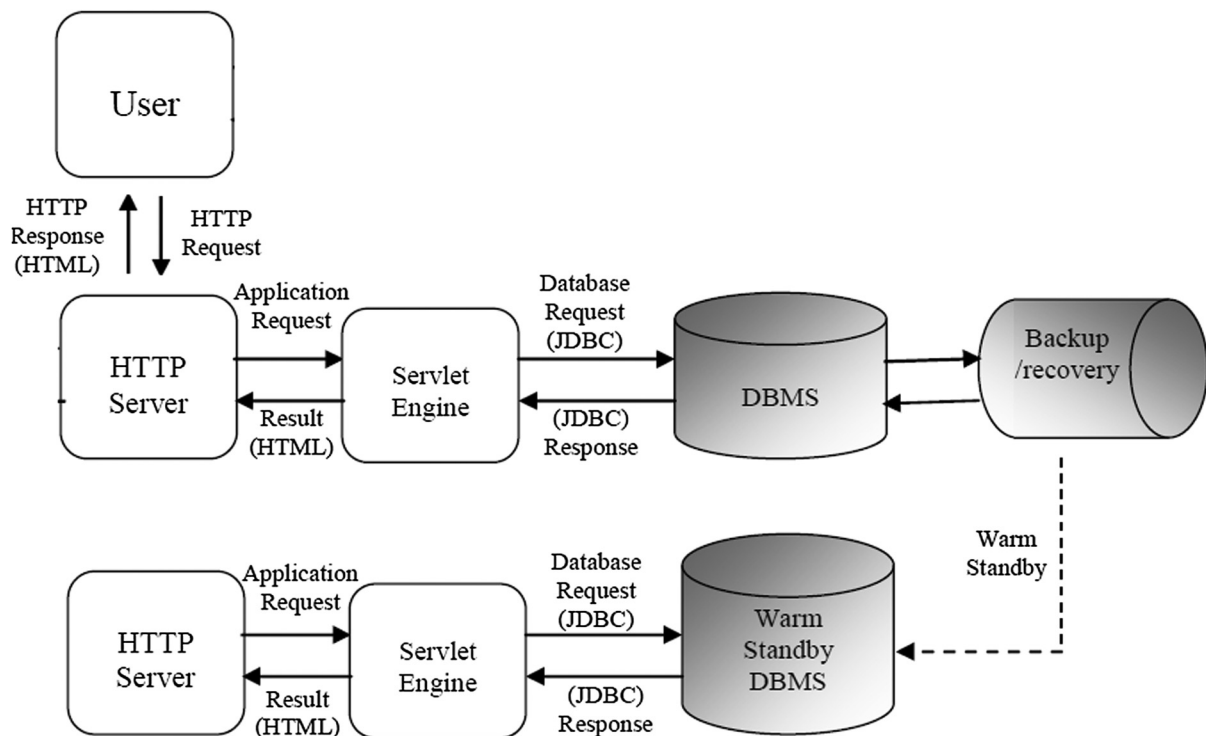


FIGURE 2 NetDB2 Architecture⁶

According to Figure 1, DOs connect to the service of SPs by using the Internet and accomplish required operations on data sets. Many authors⁶⁻¹¹ designed their own customized architectures according to the specifications of DaaS after analyzing their requirements. Figure 2 shows the custom-built DaaS architecture of Hacigumus et al⁶ named NetDB2 architecture, which is also developed according to specifications of DaaS.

NetDB2 has a three-tier architecture having three layers. In the presentation layer, session management, user authentication, and a web browser are included. Java servlets are used to manage the application layer, and it is also responsible for user authentication. The idea behind the data management layer is to manage backup/recovery servers.

Another DaaS architecture from Hacigümüş et al^{9,12,13} is shown in Figure 3 in which a query is sent out to the encrypted database. This database is also RDBMS, which is hosted by SPs using a server. That query is processed on data that is stored at the server. The duty of the client is also managing the metadata against the query for the proper representation at the server site.

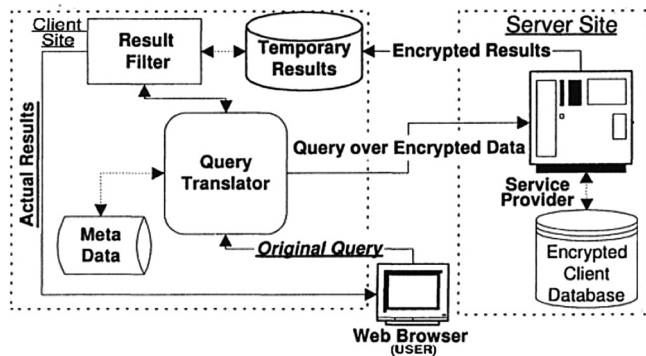


FIGURE 3 Outsourced database model⁹

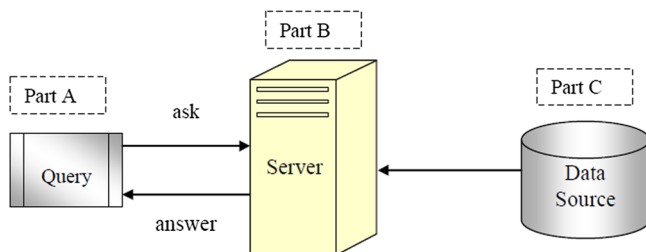


FIGURE 4 Modified NetDB2 model⁷

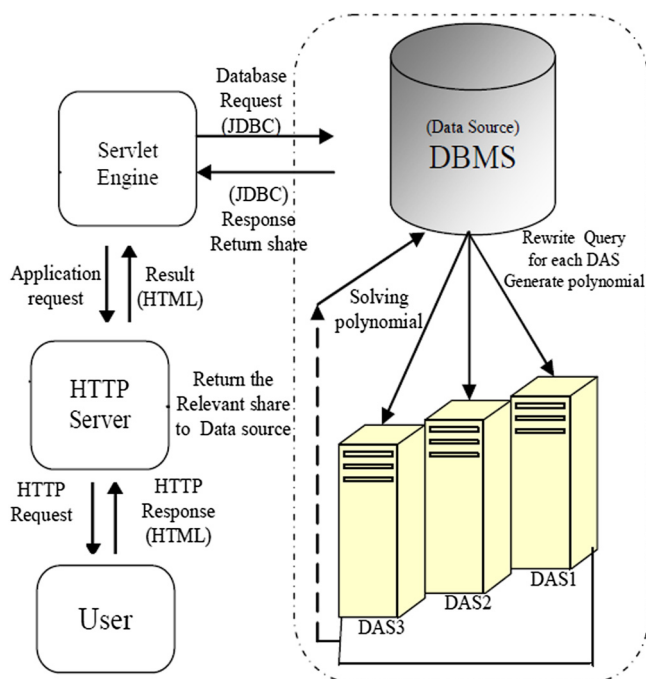


FIGURE 5 Inside Part A⁷

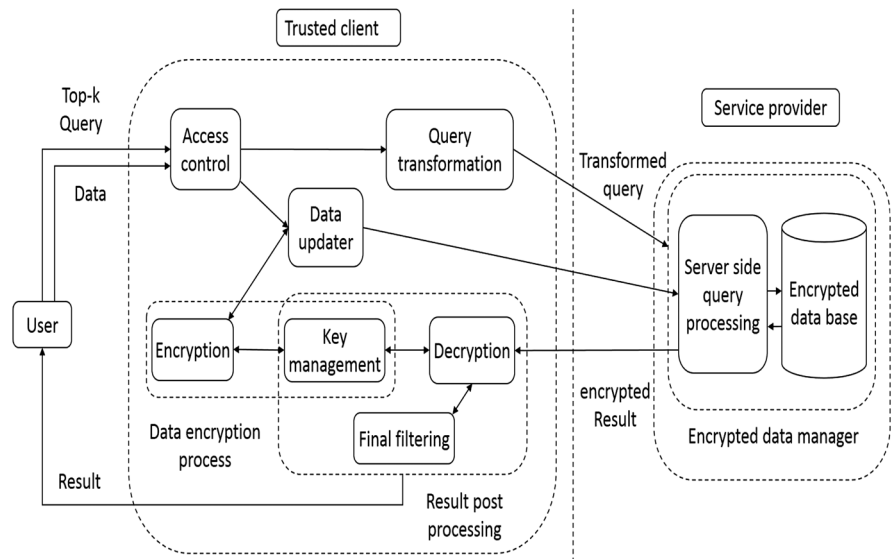


FIGURE 6 DaaS architecture with privacy¹¹

Alzain and Pardede⁷ proposed a modified version of NetDB2 architecture, which was later named as the NetDB2 multishare model. This model may be viewed in Figures 4 and 5.

The model proposed by Hacigumus et al⁶ was introduced explicitly for the purpose of privacy. This variant of the DaaS model was actually a secret sharing algorithm rather than an encryption. This model was used by the service of NetDB2. The primary focus of this model was to get rid of the high cost of encryption as well as decryption. This model has three main parts. In Figure 4, it is shown that the client communicates with the server to entertain queries, and data is stored on the server by using a data source. The server is considered a trusted server that manages encryption and decryption. This model does not offer encryption; instead, it supports the method of secret sharing.

Mahboubi et al¹¹ suggested another DaaS architecture, which provides a privacy guarantee as shown in Figure 6. This system comprises two parts: service provider and trusted client. Three activities are performed by trusted clients like encryption and decryption of data as well as managing user access. Access rights of all users are managed by a trusted client. These users run queries on DaaS. Security keys are also controlled by this part. The service provider, which is a second part, stores encrypted data and validates the execution of algorithms that are facilitated by the trust client.

3 | SECURITY AND PRIVACY OF DaaS

DaaS not only provides a group of features with lower cost but also invites new challenges like security and privacy. Moreover, another challenge is the low processing speed of data.

Will placing data on the SP site ensure security? The right answer may be “no” because there are many attacks that are associated with the connectivity of the Internet. For this reason, we cannot blindly trust the SP. One solution to data security and privacy might be achieved after performing data encryption before pushing it to the DaaS system. However, after that, the new challenge arises: How do we encrypt data and what algorithm should we use? Algorithms may decrease the data storage and retrieval speed performance. Various researchers exploited different algorithms^{6,8,13,14} to encrypt data, but the difference was minor in relation to performance.

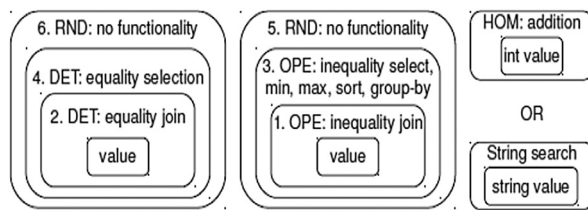
Other researchers, such as Hacigumus et al,⁶ proposed that connectivity through the Internet invites new overheads, which is responsible for the degradation of system performance. After considering this problem, they suggested a query processing mechanism in NetDB2, which was quite efficient, especially for data encryption.

According to the proposal of Chen and Zhao¹⁵ and Xia et al,¹⁶ encryption may be performed according to data granularity. It means that field, row, and table may be encrypted. Additionally, encryption is also readily available by built-in hardware features¹⁷ or through software. IBM S/390¹⁸ has a cryptographic coprocessor that works under OS/390 and has cryptographic service facility that uses ICSF libraries, which is actually a hardware feature whether IBM DB2 offered a facility named “editproc”¹⁹ for its OS/390, which is also linked with database relation. It is invoked for an entire row. Software-level encryption ensures custom as well as prebuilt tools to encrypt data before pushing into the database of DaaS.²⁰

eid	ename	Salary	Addr	did
23	Tom	70 K	Maple	40
860	Mary	60 K	Main	80
320	John	23 K	River	35
200	Sarah	55 K	River	10

TABLE 1 Before encryption⁹

RID	Etuple	eid ^{id}	ename ^{id}	Salary ^{id}	eid ^f	ename ^f
1	=*?Ew@R*[(ii = +,- ...	2	19	81	@R* ...	I(* ...
2	B*([ii(*?Ew@ = 1,r ...	4	31	59	E:q!, ...	=+, ...
3	w@ = W*([ii(*?E:j ...	7	59	22	@Aw* ...	¿[...
4	ffTi* @ = U(i?G+,a ...	8	49	59	hRs(...	?@Q = ...

TABLE 2 After encryption⁹FIGURE 7 Onion-shaped encryption²⁶

Bucketization technique^{13,21,22} is used to push data in DaaS in which partitions are prepared of a table/relation's column, and specific ID is also assigned. For every table or relation which is defined in DaaS, a separate encrypted table or relation is also constructed.

In Table 2, etuple is a string that is an encrypted form of a whole row shown in Table 1. eid^{id} shown in Table 2 is encrypted bucketization ID of eid, which is shown in Table 1. While eid^f mentioned in Table 2 is an encrypted string of eid shown in Table 1.⁹

Hacigümüş et al⁹ introduced a storage model of the encrypted database to ensure a solution to data integrity problems. Other researchers^{23,24} have worked for advanced topics on integrity. Subsequent to the storage of encrypted data, the next challenge is how to make sure if the data that we are retrieving is original; for that problem, Hacigümüş et al⁹ proposed techniques for data integrity. Integrity problems²⁵ are raised by both malicious as well as nonmalicious threats. Adversaries disrupt the security for some bad intentions, while the failure of the system may be the type of nonmalicious threat. These authors' worked to control the category nonmalicious threats.

There are two types of integrity. The first one is record-level integrity, which is used to manage the record contents. These record contents are unauthorizedly manipulated. The authors proposed the algorithm record integrity code (RIC). This RIC is entrenched with each record. After that, encryption of data is achieved on the entire record. Although the problem with record-level integrity is that it does not provide the guarantee for the insertion as well as deletion of old or new rows, there should be another type of a security-like signature-based mechanism through which tables or relations be created or updated.

Curino et al²⁶ designed a privacy design called privacy adjustable design, which was built according to the layers of the onion. In that design, each integer value was stored as thrice in this onion-shaped model and twice using schemes like deterministic encryption (DET), randomized encryption (RND), and order-preserving encryption (OPE).^{22,27,28} Moreover, homomorphic encryption (HOM) was used once.^{29,30}

The test was performed on the above proposed onion-shaped model by using CryptDB.³¹⁻³³ Basic calculations are performed on integers, so for that cause, integer values are stored three times. In Figure 7, the leftmost onion is placed for checks of equality. In the outer layer, RND is pasted, which ensures maximum privacy. It produces different ciphertexts every time, but it does not ensure computation as efficient. That was the primary purpose of the implementation of RND in the outer layer. DET is a relatively weaker encryption because it generates identical ciphertexts every time. DET was used to perform equality selection and join. Although in the center portion of the onion-shaped model, which is proposed for inequality checks, the same RND was assigned to the outer layer where the OPE mechanism was adopted for the middle layer. Encryption by the same order is generated by OPE, which makes OPE as a weaker performer, but at the same time, comparison functions on ciphertexts are performed more efficiently without decryption. On the rightmost of this onion-shaped model, HOM was used to perform aggregation functions on the ciphertexts.

Agrawal et al³⁴ proposed a novel approach to attain data privacy features. According to their proposed idea, they planned to split data on different hosts that are located in different locations and are unable to connect with each other. DOs are only allowed to connect with hosts. After establishing successful connections, DOs will collect and bind data and performed specified operations to regenerate the original shape of the data. This technique is quite capable of deceiving adversaries because data from a single host is meaningless if any adversary may be able to access a host. This technique was quite effective and not efficient because of more cost and connection delay; moreover, metadata security is not ensured in this technique.

L-EncDB was proposed by Li et al,³⁵ which has separate layers of SQL, ie, data flows and encryption layer. They utilized format-preserving encryption (FPE),³⁶ OPE, and fuzzy query encryption (FQE).^{37,38}

FPE provides excellent performance, but it is only able to perform SQL like operations directly on the unencrypted shape of data. FQE is designed to run SQL queries using “LIKE” type operator with a “%” sign. FQE is a convenient tool because we can retrieve records only after providing some characters of the required keyword. Although OPE encrypts plaintexts to ciphertexts while preserving the same order, which makes it possible to compare ciphertexts more efficiently without decryption. Additional fields were used on the database layer for OPE and FQE, and on running SQL queries like Insert or Update; they provide three possibilities that are a standard query, fuzzy query, and range queries.

Other researchers¹⁰ suggested a new proposal to ensure the privacy of stored data. They suggested using the hash function³⁹ to encrypt names of the table and the names of fields of selected tables. Their another proposal was to use the XML file that was used to store hashed form of metadata in the shape of data tags, which was helpful to hide the structure of the database. While for data, they used AES encryption.⁴⁰

4 | QUERY PROCESSING ON DaaS

Mykletun and Tsudik⁸ elaborated on the technique of query processing in the DaaS system in which users sent out a query Q to retrieve the data of employees' data whose has salaries between \$30 000 to \$75 000. After that, Query Q is transformed to Q_C and Q_S for client and server, respectively. Q_C is the query of the client, while Q_S is the translation of bucket level¹¹ of query Q . Q_S is additionally transferred into Q_{SC} . Q_{SC} denotes Query Server Certain: It has upper or lower boundary values of the bucket. Q_{SM} denotes Query Server Maybe: It shows that values will be inside the bucket. Q_{SC} and Q_{SM} then ran on the server, and obtained results are pushed back to the client, which are merged afterward. This process is shown in Figure 8.

Same as in the work of Alzain and Pardede⁷ and Agrawal et al,³⁴ other researchers proposed other shapes of queries that also capable of being executed on DaaS. Those types of queries are called Aggregation query, Range query, and Exact Match query.

Mykletun and Tsudik proposed⁸ the right approach for aggregation queries that were precompiled aggregated values. That values were Sum and Count. These were pushed into the database after encryption. Table 3 clearly shows that these values were stored in the form of a bucketization ID. Performance may also be improved by using precompiled aggregated upon data retrieval because the DaaS server would not manipulate aggregation for every client, but it needs further storage.

Usually, queries are executed once received in the ready queue from clients but in a sequential way. While the parallel execution of queries is different in which queries may be executed in parallel, which ultimately eliminates some cost.

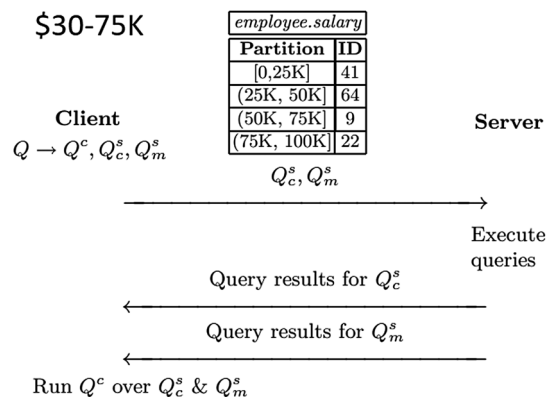
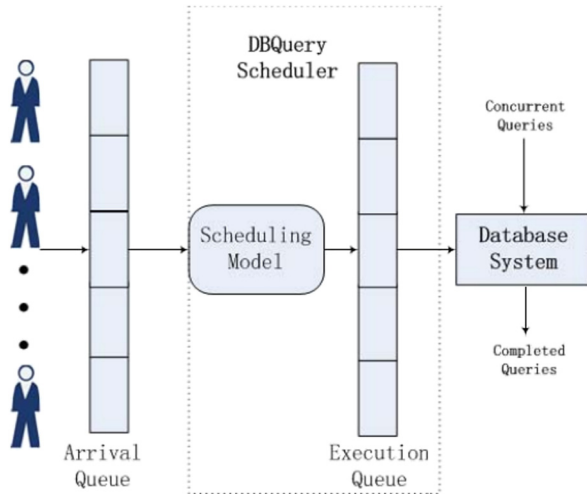


FIGURE 8 Client and server query processing⁸

<i>Employee.salary.aggregates</i>		
Bucket ID	SUM	COUNT
41	Enc(930)	Enc(15)
64	Enc(1020)	Enc(13)
9	Enc(774)	Enc(9)
22	Enc(567)	Enc(6)

TABLE 3 Precompiled aggregated values⁸FIGURE 9 QScheduler⁴¹

Query	CUDA (ms)	MySQL (Memory)	Speedup
a	10.95	126	11.5
b	12.71	206	16.2
c	179.8	1.27	7.06
d	216.4	2.57	11.88

TABLE 4 Execution time⁴²

Parallel execution of queries is a concept that is being actively investigated by numerous researchers by many years to achieve an efficient reduction in query execution time. Zhang et al⁴¹ proposed the QScheduler technique. This technique was able to work on the basis of external scheduling shown in Figure 9.

The arrival queue is filled with queries received from different clients but all at the same time. Afterward, the duty of scheduling mechanism starts by making a schedule of queries for real execution by QScheduler. In QScheduler, they proposed three types of scheduling algorithms.

FCFS is very simple in which queries are executed, which came in an execution queue. Because of the actual execution in STF, there is a need for some prior work to differentiate the least time taker query that will be executed first. To achieve this perception, execution times of queries are calculated for ranking purposes. Although in ICMF, they mixed the different queries and then put these queries to execution queue, and after analyzing the execution behavior of these mixed queries, they obtained the interaction time of the best query. However, mixing these queries needs further overhead, which ultimately degrades overall performance.

Definitely, there may also be other methods to improve further the overall performance of queries that are under execution. Another simple method may be to execute queries on arrays of GPUs or single GPU in comparison with CPU. If we compare GPUs with CPUs, then we can analyze that GPUs have hundreds of cores to perform mathematical calculations, which may work on different tasks at the same time. Huang and Chen⁴² proposed a model that was easily deployable on Nvidia's Compute Unified Device Architecture (CUDA) after using the MySQL database.

CUDA is a parallel computing environment. To work with CUDA, there is a need for a library that is available in C++ programming language, and this library may be executed by the help of Nvidia CUDA Compiler Driver.

On this CUDA mechanism, they executed different experiments on databases like sorting data, grouping data, and relation joining. According to their results which they revealed, running queries on GPUs provide much more performance than CPUs that are shown in Table 4.

TABLE 5 Query performance analysis on Encrypted Data

Research paper	Pub. Year	Hardware for Experiment	Encryption Meth. and Type	Testing Standard	CPU Time (MS)	Query Exec. Time (MS)
Providing database as service ⁶	2002	IBM Netfinity 5500 server with dual Pentium III 600 MHz 512 MB RAM 10 MB Internet Link	Blowfish with 64bit block Row Level Encryption	TPC-H	4.4	
Ensuring the Integrity of Encrypted Databases in the Database-As-a-Service Model ⁹	2004	Pentium III-750 MHz 512 MB RAM IBM DB2	Record Level Integrity (RIC) Page Level Integrity (Embedded before Encryption)	TPC-H	1.17	1.18
Integrity Auditing of Outsourced Data ⁴³	2007	Pentium IV D 2.8 GHz 512 MB RAM IBM DB2 v8.2	RSA MD5	TPC-H		10
Relational Cloud: A Database-as-a-Service for the Cloud ²⁶	2011		Onion Layers of Encryption RND, DET, OPE, HOM	TPC-C in CryptDB 100000 Select/Update		1.25 (On 15 clients)
A performance test of Query operation on an Encrypted Database ⁴⁴	2013	Intel Core i5 650 2GB Ram Windows 7, MS SQL 2008	Bloom Filter with Bucketization SHA1, MD5, SHA256 Tuple Level Encryption			0.53
L-EncDB: A lightweight framework for privacy-preserving data queries in cloud computing ³⁵	2015	Intel Core i5-3337U 1.8 GHz 4Gb Ram Windows 7	FPE, FQE, OPE			0.6
Speed up Querying Encrypted Data on Outsourced Database ¹⁰	2017	Intel Core I3 @ 1.5 GHz 4GB RAM	XML with AES	Parrallel Query Execution		(For subsequent)0.16 (For Parallel) 0.09
Parallel and Distributed Processing of Reverse Top-k Queries ⁴⁵	2019					100
Parallel query execution over encrypted data in database-as-a-service (DaaS) ⁴⁶	2019	Core i5-6200u 8GB RAM	AES	Parrallel Query Execution	On CPU idle	(For subsequent) 0.01 (For Parallel) 0.003 (For subsequent) 0.2 (For Parallel) 0.015

5 | PERFORMANCE EVALUATION OF DaaS

Although DaaS offers a bundle of facilities at the same time, there must be some schemes for the assurance of security and privacy of stored data. Several authors proposed appealing models for the protection of data. However, at this stage, another challenge arises, which is the performance of processing on encrypted data because the implementation of encryption adds overhead.

So, we have conducted a comparison of the performance of data processing on encrypted data in the context of DaaS on the state-of-the-art^{6,9,10,26,35,43-46} shown in Table 5.

Hacigumus et al⁶ proposed DB2 modified model and named it NetDB2 in which they adopted Blowfish¹³ algorithm while implementing it on row level and conducted an experiment that takes 4.4-ms CPU time at best. They used industry standard TPC-H,⁴⁷ which is a decision support benchmark and consists of business-oriented ad hoc queries majorly focusing on large volume and high complexity. Hacigumus et al,⁹ focused on the integrity of the encrypted database and introduced RIC algorithms, which are embedded on tuple before actual encryption and used as the same TPC-H standard and got some improved results like 1.18 ms of query execution. Xie et al⁴³ worked on the auditing of integrity using RSA and MD5 algorithms and tested through TPC-H and obtained results of 10 ms of query execution, which was not good enough.

Curino et al²⁶ implemented layers of encryption schemes according to the complexity of encryption schemes and named it onion layers of encryption. They implemented it on CryptDB and used TPC-C⁴⁸ platform for a test bed. TPC-C is an online transaction processing (OLTP) benchmark that simulates the complex activities in OLTP application environments focusing on multiple online sessions and simultaneous execution. In CryptDB, they executed 100 000 select/update queries and obtained impressive results like 1.25 ms. Kim et al⁴⁴ used a bloom filter with bucketization techniques and performed a test using tuple level encryption with SHA1, MD5, and SHA256 schemes. While in an experiment, they obtained a query execution time of 0.53 ms. Li et al³⁵ proposed a new scheme called L-EncDB, which was a privacy-preserving tool using a separate layer of encryption having FPE, FQE, and OPE schemes. They defined tree-shaped execution in which processing is performed according to the type of query, eg, for range queries, OPE was there, and for the fuzzy query, FQE schemes were used. While during an experiment on the select query, 0.60 ms time was recorded.

We have not seen any improvement over 0.53 ms, but Ho et al¹⁰ proposed new ideas relating to parallel query execution in which they used XML for a hiding database structure and an AES encryption scheme. They used the divided task method and performed the multithreading method up to two threads implemented in C# and obtained much improvement over previous research. They showed that only 0.16 ms was taken subsequently, and the improvement of only 0.09 ms was taken for parallel query execution.

Nikitopoulos et al⁴⁵ performed parallel and distributed processing of Top-k queries. They presented their results in which they show how many gigabytes they fetched. They used sizes of 5 GB, 10 GB, and 50 GB. Moreover, the time taken for query execution was in seconds rather than in milliseconds. Here, there was a need to convert gigabyte data into a single tuple. So, according to our database, which has 1 000 000 records having 2 GB of data. So after calculating a rough estimate, query execution time taken for their proposal parallel processing method was much higher, which was near 100 ms.

Ahmad et al⁴⁶ proved that the results of the work of Ho et al¹⁰ are not as the same as they should be. Moreover, they proved that the idea of Ho et al¹⁰ about the degradation of the multithreading approach after two threads is also not correct. Ahmad et al⁴⁶ implemented parallel query execution of up to six threads, and their result is entirely far better than the claim of Ho et al.¹⁰ They performed experiments on two scenarios: first, when the CPU was idle, and second when the CPU was busy. Even their results on busy CPU were 0.015 milliseconds, which were quite promising than that of Ho et al¹⁰ shown in Table 5.

In Table 5, the proposed results are analyzed according to CPU and query execution times, and here, these times are shown in milliseconds, which are not the same as in several research articles. We have computed these CPU and Query execution values for the single tuple. This step was taken because different authors used different parameters, and the number of rows fetched was also different. Therefore, to obtain a fair comparison, their results are converted to two main metrics, which are CPU and query execution times for a single tuple.

6 | CONCLUSION

Although we have defined architecture, security and privacy, and query processing in the context of DaaS, our primary focus was to analyze the current state of the art research works relating to the performance of query processing on encrypted data.

We have concluded that performance query performance over encrypted data was enhanced up to some extent. However, after the application of parallel query processing, researchers used multithreading techniques up to two threads, and performance was much enhanced.

7 | LIMITATIONS AND FUTURE WORKS

This work is limited to the query performance of DaaS. Another essential metric, security, is not critically analyzed rather than some introduction. This is the limitation of this work.

Thus, for future work, we have planned to critically analyze and provide a solution to some security threats to DaaS after implementing different encryption and decryption techniques.

8 | SUGGESTIONS

Security of DaaS is critical because data of DOs is stored on the SP site. Therefore, there should be some mechanism for encryption and decryption, but another challenge may arise here, such as cracking using brute-force type attacks, which may be tackled with password security like AES; however, another issue arises, which occurs in textual passwords. This attack may be called as observation or shoulder surfing attacks. Therefore, for the suggestion, graphical authentication schemes which work best against shoulder surfing attacks⁴⁹ must be followed.

Moreover, DaaS is no doubt a futuristic architecture, but it needs high-speed internet connection; otherwise, performance will be further degraded. Therefore, for suggestions from our side, this architecture will be better to implement in developed countries where internet connection speeds are functional, unlike in developing countries.

ORCID

Muhammad Asif  <https://orcid.org/0000-0003-1839-2527>

REFERENCES

1. Bendre MR, Thool VR. Analytics, challenges and applications in big data environment: a survey. *J Manag Anal*. 2016;3(3):206-239.
2. Skourletopoulos G, Mavromoustakis CX, Mastorakis G, et al. Big data and cloud computing: a survey of the state-of-the-art and research challenges. In: *Advances in Mobile Cloud Computing and Big Data in the 5G Era*. Cham, Switzerland: Springer; 2017:23-41.
3. Yang C, Huang Q, Li Z, Liu K, Hu F. Big data and cloud computing: innovation opportunities and challenges. *Int J Digit Earth*. 2017;10(1):13-53.
4. Olle TW. Database management system (DBMS). In: *Encyclopedia of Computer Science*. Hoboken, NJ: John Wiley and Sons Ltd; 2003.
5. Sumathi S, Esakkirajan S. Structured query language. In: *Fundamentals of Relational Database Management Systems*. 2007:111-212.
6. Hacigumus H, Iyer B, Mehrotra S. Providing database as a service. In: *Proceedings of the 18th International Conference on Data Engineering*; 2002; San Jose, CA.
7. Alzain MA, Pardede E. Using multi shares for ensuring privacy in database-as-a-service. Paper presented at: 2011 44th Hawaii International Conference on System Sciences; 2011; Kauai, HI.
8. Mykletun E, Tsudik G. Aggregation queries in the database-as-a-service model. In: *Data and Applications Security XX: 20th Annual IFIP WG 11.3 Working Conference on Data and Applications Security, Sophia Antipolis, France, July 31-August 2, 2006. Proceedings*. Berlin, Germany: Springer Science + Business Media; 2006:89-103.
9. Hacigumus H, Iyer B, Mehrotra S. Ensuring the integrity of encrypted databases in the database-as-a-service model. In: *Data and Applications Security XVII*. Berlin, Germany: Springer Science + Business Media; 2004:61-74.
10. Ho KG, Vu L, Nguyen NH, Nguyen HM. Speed up querying encrypted data on outsourced database. In: *Proceedings of the 2017 International Conference on Machine Learning and Soft Computing*; 2017; Ho Chi Minh City, Vietnam.
11. Mahboubi S, Akbarinia R, Valduriez P. *Top-k Query Processing Over Outsourced Encrypted Data*. Research Report. Sophia Antipolis, France. INRIA Sophia Antipolis-Méditerranée; 2017.
12. Hacigumus VH. *Privacy in Database-as-a-Service Model*. Irvine, CA: University of California, Irvine; 2003.
13. Hacigumus H, Iyer B, Li C, Mehrotra S. Executing SQL over encrypted data in the database-service-provider model. In: *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*; 2002; Madison, WI.
14. Bilal M, Asif M, Bashir A. Assessment of secure OpenID-based DAAA protocol for avoiding session hijacking in web applications. *Secur Commun Netw*. 2018;2018.
15. Chen D, Zhao H. Data security and privacy protection issues in cloud computing. Paper presented at: 2012 International Conference on Computer Science and Electronics Engineering; 2012; Hangzhou, China.

16. Xia X, Li Y, Xia Z, Wang R. Data encryption based on multi-granularity reversible cellular automata. Paper presented at: 2009 International Conference on Computational Intelligence and Security; 2009; Beijing, China.
17. Yang B, Wu K, Karri R. Scan based side channel attack on dedicated hardware implementations of data encryption standard. Paper presented at: International Test Conference; 2004; Charlotte, NC.
18. Webb CF. IBM z10: the next-generation mainframe microprocessor. *IEEE Micro*. 2008;28(2):19-29.
19. Bruni P, Becker P, Favero W, et al. *Optimizing DB2 Queries With IBM DB2 Analytics Accelerator for z/OS*. Armonk, NY: IBM Redbooks; 2012.
20. Gul S, Asif M, Ahmad S, Yasir M, Majid M, Malik M. A survey on role of Internet of Things in education. *Int J Comput Sci Netw Secur*. 2017;17(5):159-165.
21. Bellare M, Boldyreva A, O'Neill A. Deterministic and efficiently searchable encryption. In: *Advances in Cryptology-CRYPTO 2007: 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007. Proceedings*. Berlin, Germany: Springer-Verlag Berlin Heidelberg; 2007:535-552.
22. Agrawal R, Kiernan J, Srikant R, Xu Y. Order preserving encryption for numeric data. In: *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*; 2004; Paris, France.
23. Sivathanu G, Wright CP, Zadok E. Ensuring data integrity in storage: techniques and applications. In: *Proceedings of the 2005 ACM Workshop on Storage Security and Survivability*; 2005; Fairfax, VA.
24. Martinenghi D. *Advanced Techniques for Efficient Data Integrity Checking* [PhD thesis]. Roskilde, Denmark: Department of Computer Science, Roskilde University; 2005.
25. Huth CL, Chadwick DW, Claycomb WR, You I. Guest editorial: a brief overview of data leakage and insider threats. *Inf Syst Front*. 2013;15(1):1-4.
26. Curino C, Jones EP, Popa RA, et al. Relational cloud: a database-as-a-service for the cloud. 2011.
27. Popa RA, Li FH, Zeldovich N. An ideal-security protocol for order-preserving encoding. Paper presented at: 2013 IEEE Symposium on Security and Privacy; 2013; Berkeley, CA.
28. Boldyreva A, Chenette N, Lee Y, O'Neill A. Order-preserving symmetric encryption. In: *Advances in Cryptology - EUROCRYPT 2009: 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings*. Vol. 5479. Berlin, Germany: Springer-Verlag Berlin Heidelberg; 2009:224-241.
29. Gentry C. Fully homomorphic encryption using ideal lattices. In: *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*; 2009; Bethesda, MD.
30. van Dijk M, Gentry C, Halevi S, Vaikuntanathan V. Fully homomorphic encryption over the integers. In: *Advances in Cryptology - EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30 - June 3, 2010. Proceedings*. Berlin, Germany: Springer; 2010:24-43.
31. Popa RA, Redfield C, Zeldovich N, Balakrishnan H. CryptDB: protecting confidentiality with encrypted query processing. In: *Proceedings of the 23rd ACM Symposium on Operating Systems Principles*; 2011; Cascais, Portugal.
32. Popa RA, Redfield C, Zeldovich N, Balakrishnan H. CryptDB: processing queries on an encrypted database. *Commun ACM*. 2012;55(9):103-111.
33. Popa RA, Zeldovich N, Balakrishnan H. *CryptDB: A Practical Encrypted Relational DBMS*. Technical Report. Cambridge, MA: MIT Computer Science and Artificial Intelligence Laboratory; 2011.
34. Agrawal D, El Abbadi A, Emekci F, Metwally A. Database management as a service: challenges and opportunities. Paper presented at: 2009 IEEE 25th International Conference on Data Engineering; 2009; Shanghai, China.
35. Li J, Liu Z, Chen X, Xhafa F, Tan X, Wong DS. L-EncDB: a lightweight framework for privacy-preserving data queries in cloud computing. *Knowl Based Syst*. 2015;79:18-26.
36. Bellare M, Ristenpart T, Rogaway P, Stegers T. Format-preserving encryption. In: *Selected Areas in Cryptography: 16th Annual International Workshop, SAC 2009, Calgary, Alberta, Canada, August 13-14, 2009, Revised Selected Papers*. Vol. 5867. Berlin, Germany: Springer-Verlag Berlin Heidelberg; 2009:295-312.
37. Li J, Wang Q, Wang C, Cao N, Ren K, Lou W. Fuzzy keyword search over encrypted data in cloud computing. In: *Proceedings of the IEEE INFOCOM*; 2010; San Diego, CA.
38. Fu Z, Wu X, Guan C, Sun X, Ren K. Toward efficient multi-keyword fuzzy search over encrypted outsourced data with accuracy improvement. *IEEE Trans Inf Forensics Secur*. 2016;11(12):2706-2716.
39. Khan FA, Ahmad A, Imran M, Alharbi M, Jan B. Efficient data access and performance improvement model for virtual data warehouse. *Sustain Cities Soc*. 2017;35:232-240.
40. Sher A, Javaid N, Azam I, et al. Monitoring square and circular fields with sensors using energy-efficient cluster-based routing for underwater wireless sensor networks. *Int J Distrib Sens Netw*. 2017;13(7):1550147717717189.
41. Zhang Q, Li S, Xu J. QScheduler: a tool for parallel query processing in database systems. Paper presented at: 2014 19th International Conference on Engineering of Complex Computer Systems (ICECCS); 2014; Tianjin, China.
42. Huang Y-F, Chen W-C. Parallel query on the in-memory database in a CUDA platform. Paper presented at: 2015 10th International Conference on P2P, Parallel, Grid, Cloud and Internet Computing (3PGCIC); 2015; Kraków, Poland.
43. Xie M, Wang H, Yin J, Meng X. Integrity auditing of outsourced data. In: *Proceedings of the 33rd International Conference on Very Large Data Bases*; 2007; Vienna, Austria.
44. Kim JH, Sahama T, Kim SY. A performance test of query operation on encrypted database. In: *Future Information Communication Technology and Applications*. Dordrecht, The Netherlands: Springer Science+Business Media; 2013:801-810.

45. Nikitopoulos P, Sfyris GA, Vlachou A, Doukeridis C, Telelis O. Parallel and distributed processing of reverse top-k queries. Paper presented at: 2019 IEEE 35th International Conference on Data Engineering (ICDE); 2019; Macao.
46. Ahmad A, Ahmad M, Habib MA, et al. Parallel query execution over encrypted data in database-as-a-service (DaaS). *J Supercomput*. 2019;75(4):2269-2288.
47. Khan FA, Khan M, Asif M, Khalid A, Haq IU. Hybrid and multi-hop advanced zonal-stable election protocol for wireless sensor networks. *IEEE Access*. 2019;7:25334-25346.
48. Khan FA, Han Y, Pillana S, Brezany P. Provenance support for grid-enabled scientific workflows. Paper presented at: 2008 Fourth International Conference on Semantics, Knowledge and Grid; 2008; Beijing, China.
49. Ahmad A, Asif M, Hanif MK, Talib R. Secure graphical password techniques against shoulder surfing and camera based attacks. *Int J Comput Sci Inf Secur*. 2016;14(10):808.

How to cite this article: Khan FA, Jamjoom M, Ahmad A, Asif M. An analytic study of architecture, security, privacy, query processing, and performance evaluation of database-as-a-service. *Trans Emerging Tel Tech*. 2022;33:e3814. <https://doi.org/10.1002/ett.3814>